
Topic 4 — Async Research Assistant

M503

Orkhan Nuriyev (@orxannuriyev) • Nazrin Burziyeva (@nazrinburz) • Gunel Garayzade (@gunelrafig)

Final Project Defense – 2026-09-19

github.com/orxannuriyev/research_assistant

The problem in one sentence

What the system does

A user asks a natural-language question; we query Wikipedia, arXiv, and Web concurrently, and return a single synthesised answer with numbered inline citations.

Inputs

- ▶ Natural-language question (≤ 500 chars)
- ▶ Target source list (e.g. wiki, arxiv, web)

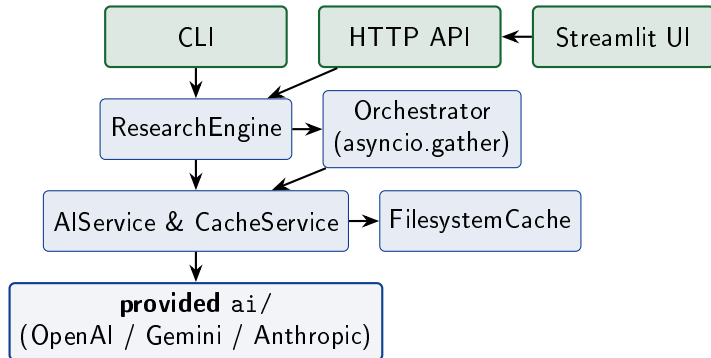
Outputs

- ▶ Synthesised answer text
- ▶ Numbered references used (title, url)

Entry points: CLI (`python -m researcher ask`) & HTTP API (`POST /research`)

Architecture

Module map. The boundary between the *provided* ai/ package and *our* SE layer is shown explicitly.



ResearchEngine coordinates the concurrent fetching via *Orchestrator* and the final synthesis via *AIService*.

Three design decisions we can defend

1. **asyncio over threads**

The workload is purely I/O-bound (network latency); threads have higher memory overhead and GIL contention.

2. **Filesystem JSON cache over SQLite**

Atomic JSON writes (temp file + `os.replace`) prevent corruption and are trivial to inspect during debugging.

3. **Pydantic Settings over manual env parsing**

Provides built-in type casting, defaults, and validation for our 12 configuration variables with a single `Settings` singleton.

Concurrency: the benchmark

Workload: benchmark query across the three source adapters; Wikipedia returned 403 in the recorded run.

	Sequential	Concurrent (sem=5)
Wall-clock (s)	3.03	2.38
Speedup	1.0×	1.28×
Provider 429s	0	0

Bottleneck named: arXiv's mandatory 1s polite interval limits parallel gains.

What didn't reach 3×: The final LLM synthesis step is fundamentally sequential, and the arXiv lock limits throughput.

Numbers run on Windows, Python 3.11 via `python scripts/bench.py`.

Robustness: one concrete failure mode

Scenario: Wikipedia intermittently returned HTTP 403 Forbidden due to IP blocking.

How we surfaced it: During live testing of the concurrent fetch pipeline.

What the system does:

- ▶ **Retry:** Tenacity retries HTTP 4xx (non-429) errors up to 3 times with exponential backoff.
- ▶ **Log:** `WARNING source 'wikipedia' failed: 403 Forbidden - skipping.`
- ▶ **Degrade:** `asyncio.gather` receives empty results for Wikipedia.
- ▶ **Validate:** The engine successfully synthesises an answer using only arXiv and Web without crashing.

Live demo

What we will run

An offline question answering pipeline using mock AI endpoints and cached sources.

Demo command (works inside Docker):

```
$ docker run --rm finalproj  
$ docker run --rm --env-file .env finalproj \  
  python -m researcher ask "What is CRISPR?" --sources wiki,arxiv
```

- ▶ Outputs a cohesive markdown-styled answer with numbered citations.
- ▶ CLI correctly respects the `-sources` filter and `-no-cache` flags.

HTTP endpoint also available via `POST /research`.

Testing

Coverage & shape

- ▶ Total src/ coverage: **86%**
- ▶ Provided ai/ smoke tests: **passing**
- ▶ Test count: 51 passed
- ▶ All tests offline; AI module fully mocked

Three tests worth calling out

- ▶ Happy-path end-to-end integration
- ▶ Concurrency source failure isolation
- ▶ Cache TTL expiry catches timezone bugs

Tooling

`pytest + pytest-cov + unittest.mock.`

Limitations & what we'd do next

Where the system is brittle today

- ▶ No multi-provider failover; if the selected provider is down, we return no answer.
- ▶ arXiv's 1s lock caps throughput severely on large concurrent batches.
- ▶ Cache staleness is TTL-based; no event-driven invalidation.

Given another week we would

1. Add automatic fallback between configured LLM providers.
2. Implement a Token-Bucket rate limiter for arXiv to allow bursts.
3. Stream LLM tokens via `async for` for better perceived UI latency.

Team contributions

Member	Primary ownership	Workload
Orkhan Nuriyev	Core orchestration, storage, config	2642 added, 16 deleted, 5 commits
Nazrin Burziyeva	Service integration, CLI, tests, Docker	1278 added, 542 deleted, 13 commits
Gunel Garayzade	Initial AI/API integration, Streamlit UI, provider selection, Swagger	1395 added, 373 deleted, 6 commits

Collaborative integration: Gunel established the initial AI/API integration, completed the Streamlit UI and provider selection, and contributed PR #15's bilingual UI/API iteration; Nazrin completed the surrounding engine and service integration, robustness, tests, and Docker packaging.

AI tool use: *AI scaffolded API routes and tests; we refactored OOP structure, fixed timezone bugs, and wrote edge-case tests.*

Questions?

M503

`github.com/orxannuriyev/research_assistant`

“We can walk through any file in the repo.”